BAN HỌC TẬP CÔNG NGHỆ PHẦN MỀM

TRAINING CUỐI KỲ HỌC KỲ II NĂM HỌC 2024 – 2025

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

Sharing is learning

BAN HỌC TẬP

Khoa Công nghệ Phần mềm

Trường Đại học Công nghệ Thông tin

Đại học Quốc gia thành phố Hồ Chí Minh

CONTACT

bht.cnpm.uit@gmail.com

fb.com/bhtcnpm

fb.com/groups/bht.cnpm.uit

TEAM TIẾNG ANH

english.with.bht@gmail.com

creative.owl.se

english.with.bht

TRAINING

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Thời gian: 19:00 thứ 4 ngày 11/6/2025

Địa điểm: MS Teams – Code: w2dsy1q

Trainers: Đặng Võ Tuấn Tài – KTPM2024.3

Nguyễn Khả An – KTPM2024.1

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

I. Tổng quan về OOP

II. Lớp và đối tượng

III. Hàm bạn và lớp bạn

IV. Đa năng hóa toán tử

V. Tính kế thừa

VI. Tính đa hình

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Chương 1

TỔNG QUAN VỀ OOP

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. TỔNG QUAN VỀ OOP

1. Một số khái niệm cơ bản về OOP

- **Lập trình hướng đối tượng** là phương pháp lập trình lấy đối tượng làm nền tảng để xây dựng chương trình.
- Một chương trình hướng đối tượng sẽ bao gồm tập các đối tượng có quan hệ với nhau.
- Lớp là một mô tả trừu tượng của nhóm các đối tượng cùng bản chất, ngược lại mỗi một đối tượng là một thể hiện cụ thể cho những mô tả trừu tượng đó.

  + **Lớp** là cái ta thiết kế và lập trình.

  + **Đối tượng** là cái ta tạo (từ một lớp) tại thời gian chạy.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. TỔNG QUAN VỀ OOP

1. Một số khái niệm cơ bản về OOP

- **Thuộc tính (Attribute):** Một thành phần của đối tượng, có giá trị nhất định cho mỗi đối tượng tại mỗi thời điểm trong hệ thống.
- **Thao tác (Operation):** Thể hiện hành vi của một đối tượng tác động qua lại với các đối tượng khác hoặc với chính nó.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. TỔNG QUAN VỀ OOP

1. Một số khái niệm cơ bản về OOP

...

- Mỗi thao tác trên một lớp đối tượng cụ thể tương ứng với một cài đặt cụ thể. Một cài đặt như vậy được gọi là một **phương thức (method).**
- Cùng một phương thức có thể được áp dụng cho nhiều lớp đối tượng khác nhau. Một thao tác như vậy được gọi là có **tính đa hình (polymorphism).**

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. TỔNG QUAN VỀ OOP

2. Các đặc điểm quan trọng của OOP

- **Tính đóng gói – Encapsulation:** Đóng gói là nhóm những gì có liên quan với nhau làm một để sau này có thể dùng một cái tên để gọi đến. Dùng để che một số thông tin và chi tiết cài đặt nội bộ để bên ngoài không nhìn thấy.
- **Tính trừu tượng - Abstraction:** Trừu tượng là cách nhìn khái quát hóa về một tập các đối tượng có chung các đặc điểm được quan tâm (bỏ qua những chi tiết không cần thiết).

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. TỔNG QUAN VỀ OOP

2. Các đặc điểm quan trọng của OOP

- **Tính kế thừa - Inheritance:** Là cơ chế cho phép lớp con kế thừa các thuộc tính và phương thức từ lớp cha. Điều này giúp tái sử dụng mã nguồn và xây dựng các mối quan hệ phân cấp giữa các lớp.
- **Tính đa hình – Polymorphism:** Là cơ chế cho phép sử dụng các phương thức cùng tên nhưng với cách thức thực hiện khác nhau trong các lớp khác nhau. Điều này cho phép gọi cùng một phương thức và có kết quả khác nhau tuỳ thuộc vào đối tượng gọi phương thức đó.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. TỔNG QUAN VỀ OOP

3. Các ưu điểm của OOP

- **Tính kế thừa** giúp loại bỏ những đoạn chương trình lặp lại và mở rộng khả năng sử dụng các lớp đã được xây dựng.
- **Tính đóng gói, che dấu thông tin:** Chương trình không bị thay đổi bởi những đoạn chương trình khác.
- **Mô phỏng thế giới thực tốt hơn:** ánh xạ các đối tượng của bài toán vào đối tượng của chương trình.
- Hệ thống hướng đối tượng **dễ mở rộng, nâng cấp** thành hệ thống lớn hơn.

BAN HỌC TẬP
Sharing is learning

Chương 2

LỚP VÀ ĐỐI TƯỢNG

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

1. Cú pháp khai báo lớp

```
class [ten_class] {
    private / protected :
        //Thành phần dữ liệu
    public:
        //Thành phần xử lý
};
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

1. Cú pháp khai báo lớp

```
class [ten_class] {
    private:
    <Khai báo thành phần riêng trong từng đối tượng>
    protected:
    <Khai báo thành phần riêng trong từng đối tượng, có thể truy cập từ lớp dẫn xuất>
    public:
    <Khai báo thành phần công cộng>
};
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

2. Các thành phần của lớp

Một lớp đối tượng bao gồm 2 thành phần chính:

- **Thành phần dữ liệu (data member)**, hay còn được gọi là thuộc tính (attribute).
- **Hàm thành phần (member function)**, còn có tên gọi khác là phương thức (method), là các hành động mà đối tượng của lớp có thể thực hiện.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

3. Khai báo và tạo lập đối tượng

– Khai báo và tạo đối tượng:

```
<Tên lớp> <Tên đối tượng>;
```

– Gọi hàm thành phần của lớp:

```
<Tên đối tượng>.<tên hàm thành phần>(<DS các tham số nếu có>);
<Tên con trỏ đối tượng>→<tên hàm thành phần>(<DS các tham số nếu có>);
```

BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

3. Khai báo và tạo lập đối tượng

Ví dụ:

```
class cPhanSo {
private:
    int tu;
    int mau;
public:
    void RutGon();
};

int main(){
    cPhanSo ps;
    ps.RutGon();
    return 0;
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

4. Phạm vi truy xuất

– Trong định nghĩa của lớp ta có thể xác định khả năng truy xuất thành phần của một lớp nào đó từ bên ngoài phạm vi lớp.

– **private, protected** và **public** là các từ khóa xác định phạm vi truy xuất.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

4. Phạm vi truy xuất

– Phạm vi truy cập **public**: có thể được truy cập ở **tất cả các phần của chương trình**.

– Phạm vi truy cập **private**: chỉ có thể được truy cập **bên trong phạm vi của lớp**.

– Phạm vi truy cập **protected**: Là sự kết hợp giữa **public** và **private**, có thể được truy cập ở lớp dẫn xuất trong khi vẫn giới hạn quyền truy cập từ người dùng.

BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

4. Phạm vi truy xuất

Lưu ý:

– Một lớp có thể có nhiều nhãn private, public hoặc protected

– Mỗi nhãn này có phạm vi ảnh hưởng cho đến khi gặp một nhãn kế tiếp hoặc hết khai báo lớp.

– Nhãn private đầu tiên có thể bỏ qua vì **C++** ngầm hiểu rằng các thành phần trước nhãn public đầu tiên là private.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

5. Phương thức thiết lập - Constructor

- Các phương thức thiết lập (Constructors): có nhiệm vụ thiết lập các thông tin ban đầu của các đối tượng thuộc về lớp đối tượng khi đối tượng được khai báo.
- Bất kỳ một đối tượng nào được khai báo đều phải sử dụng một hàm thiết lập để khởi tạo các giá trị thành phần của đối tượng.
- Hàm thiết lập được khai báo giống như một phương thức với tên phương thức trùng với tên lớp và không có giá trị trả về (kể cả void).
- Constructor phải có thuộc tính `public.`

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

5. Phương thức thiết lập - Constructor

Phân biệt các loại **Constructor:**

Phương thức thiết lập mặc định (**Default Constructors**): là phương thức thiết lập các thông tin ban đầu cho đối tượng về lớp bằng những giá trị mặc định (do người lập trình quyết định).

Ví dụ:

```
PhanSo::PhanSo(){
    tu = 0;
    mau = 1;
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

5. Phương thức thiết lập - Constructor

Phân biệt các loại **Constructor:**

*Phương thức thiết lập nhận tham số đầu vào (***Paremeterired Constructors***):* Là thiết lập thông tin ban đầu cho các đối tượng thông qua các tham số đầu vào. **Constructor** nhận tham số đầu vào có thể sử dụng để ép kiểu.

Ví dụ:

```
PhanSo::PhanSo(int a, int b){
    tu = a;
    mau = b;
}
```

BOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

5. Phương thức thiết lập - Constructor

Phân biệt các loại **Constructor:**

Phương thức thiết lập sao chép (**Copy Constructors**): là phương thức nhận tham số đầu vào là 1 đối tượng cùng thuộc lớp.

Thông tin ban đầu của đối tượng sẽ giống hoàn toàn thông tin đối tượng tham số đầu vào.

Ví dụ:

```
PhanSo::PhanSo(const PhanSo& a){
    tu = a.tu;
    mau = a.mau;
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

6. Phương thức hủy bỏ - Destructor

- Các phương thức phá hủy (**Destructor**): có nhiệm vụ thu hồi lại các tài nguyên cấp phát cho đối tượng khi đối tượng hết phạm vi hoạt động.
- Destructor được gọi ngay trước khi một đối tượng bị thu hồi.
- Một lớp chỉ có ***duy nhất*** một Destructor .
- Destructor trùng tên với tên lớp nhưng có dấu ~ đặt trước .
- Được tự động gọi thực hiện khi đối tượng hết phạm vi sử dụng.
- Destructor phải có thuộc tính **public.**

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

7. Phương thức truy vấn, cập nhật

a. Phương thức truy vấn

- Có nhiều loại câu hỏi truy vấn có thể:

  - Truy vấn đơn giản ("giá trị của x là bao nhiêu?")
  - Truy vấn điều kiện ("thành viên x có > 50 không?")
  - Truy vấn dẫn xuất ("tổng giá trị của các thành viên x và y là bao nhiêu?")

- Đặc điểm quan trọng của phương thức truy vấn là nó không nên thay đổi trạng thái hiện tại của đối tượng.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

7. Phương thức truy vấn, cập nhật

a. Phương thức truy vấn

- Quy ước đặt tên: Tiền tố “get”, tiếp theo là tên của thành viên cần truy vấn.

```
int getX();
int getSize();
```

- Các loại truy vấn khác nên có tên có tính mô tả.

- Truy vấn điều kiện nên có tiền tố “is”.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

7. Phương thức truy vấn, cập nhật

b. Phương thức cập nhật

- Thường để thay đổi trạng thái của đối tượng bằng cách sửa đổi một hoặc nhiều thành viên dữ liệu của đối tượng đó.
- Dạng đơn giản nhất là gán một giá trị nào đó cho một thành viên dữ liệu.
- Đối với dạng cập nhật đơn giản, quy ước đặt tên như sau: Dùng tiền tố "set" kèm theo tên thành viên cần sửa.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

8. Thành viên tĩnh – static member

a. Thành viên static

Các thành phần tĩnh (static member) là các thành phần thuộc về cả một **lớp** chứ không thuộc về một **đối tượng cụ thể**. Điều này có nghĩa là tất cả các đối tượng của một lớp đều chia sẻ chung một thành phần tĩnh, và nó có thể được truy cập mà không cần thông qua một đối tượng.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

8. Thành viên tĩnh – static member

a. Thành viên static

- Có hai loại thành phần tĩnh:

  - **Các thuộc tính (dữ liệu thành viên) tĩnh** (Static data member).
  - **Hàm thành viên (phương thức) tĩnh** (Static member function).

- Thuộc tính tĩnh không phải là thuộc tính của một đối tượng nào cả.

- Một hàm thành viên tĩnh không gắn với bất kì đối tượng nào.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

8. Thành viên tĩnh – static member

a. Thành viên static

- Đối với *class*, ***static*** dùng để khai báo thành viên dữ liệu dùng chung cho mọi thể hiện của lớp.

- Một bản duy nhất tồn tại trong suốt quá trình chạy của chương trình.

- Dùng chung cho tất cả các thể hiện của lớp. Bất kể lớp đó có bao nhiêu thể hiện.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

8. Thành viên tĩnh – static member

b. Khởi tạo static member

Muốn khởi tạo một thành viên tĩnh chúng ta sẽ thêm từ khóa `static` vào trước dòng khai báo của nó. Như các thành viên khác thì thành viên tĩnh cũng có thể được khai báo như là một thành phần `private`, `public` hay `protected`.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

8. Thành viên tĩnh – static member

b. Khởi tạo static member

Ví dụ:

```cpp
#include <iostream>
using namespace std;
class HocSinh
{
    public:
        static int count;
        int count1 = 0;
        HocSinh() {
            ++count;
            ++count1;
        }
};
int HocSinh::count = 0;
```

```cpp
int main() {
    HocSinh h1, h2;
    cout << h1.count << " " << h1.count1 << endl;
    cout << h2.count << " " << h2.count1 << endl;
}
```

Kết quả chạy: ???

2 1

2 1

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LỚP VÀ ĐỐI TƯỢNG

8. Thành viên tĩnh – static member

b. Khởi tạo static member

Ví dụ:

```cpp
#include <iostream>
using namespace std;
class HocSinh
{
    public:
        static int count;
        int count1 = 0;
        HocSinh() {
            ++count;
            ++count1;
        }
};
int HocSinh::count = 0;
```

```cpp
int main() {
    HocSinh h1, h2;
    cout << h1.count << " " << h1.count1 << endl;
    h1.count = 100;
    cout << h2.count << " " << h2.count1 << endl;
}
```

Kết quả chạy: ???

2 1

100 1

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Chương 3

HÀM BẠN VÀ LỚP BẠN

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

HÀM BẠN

- Hàm bạn là hàm có thể truy cập thành phần private hoặc protected của lớp xem nó là bạn.
- Cú pháp: `friend <Kiểu_trả_về><Tên_hàm><DS tham số>`
- Lưu ý: hàm bạn không phải là hàm thành phần của lớp. Không cần toán tử phạm vi "::" trước tên hàm khi định nghĩa bên ngoài lớp
- Lời gọi hàm bạn giống như lời gọi hàm thông thường

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

HÀM BẠN

Ví dụ:

```
class ViDu1{
private :
    // Các thành phần
public:
    friend void f(){
        // Code
    }
};
/* Cài đặt bên trong
thân lớp */
```

```
class ViDu2{
private :
    // Các thành phần
public:
    friend void f();
};

void f(){
    // code
}
/* Cài đặt bên ngoài
thân lớp */
```

BOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

HÀM BẠN

Một hàm **f** có thể là bạn của nhiều lớp. Khi đó:

+ Khai báo **f** trong thân tất cả các class mà nó là bạn.

+ Cài đặt f bên ngoài tất cả các class.

```
class A{ friend void f();...};
class B{ friend void f();...};
class C{ friend void f();...};

void f(){
    // code
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

LỚP BẠN

- Lớp bạn là lớp có thể truy cập thành phần private hoặc protected của lớp xem nó là bạn.

- Cú pháp:

```
class A {
   private:
      // cac thanh phan
   public:
      friend class B ;
};
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

LỚP BẠN

Ví dụ:

```
class PhanSo {
private:
    int mau,tu ;
public:
    friend class MayTinh;
};

class MayTinh {
public:
    double GiaTriThapPhan(PhanSo A){
        return double(A.tu)/A.mau;
    }
};
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

LỚP BẠN

KHÔNG CÓ TÍNH ĐỐI XỨNG

- Class B là bạn của class A

-> Class A là không phải bạn của class B

KHÔNG CÓ TÍNH BẮC CẦU

- Class A là bạn của class B, class B là bạn của class C

-> Class A không phải bạn của C

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Chương 4

ĐA NĂNG HÓA TOÁN TỬ

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

ĐA NĂNG HÓA TOÁN TỬ

4.1 Giới thiệu

- Định nghĩa lại toán tử đã có trên các lớp đối tượng do người dùng tự định nghĩa để dàng thực hiện các câu lệnh trong chương trình.
- **Ví dụ:**

```
Vector2D v1(2,3);
Vector2D v2(4,5);
Vector2D ketQua;
ketQua = v1.Cong(v2); // Dùng hàm
ketQua = v1 + v2;  // Dùng toán tử "+"
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

ĐA NĂNG HÓA TOÁN TỬ

4.1 Giới thiệu

- Các loại toán tử có thể nạp chồng:

+	-	*	/	%	^	&	|
~	!	=	<	>	+=	-=	*=
/=	%=	^=	&=	|=	<<	>>	>>=
<<=	==	!=	<=	>=	&&	||	++
_	->*	,	->	[]	()	new	delete
new[]	delete[]						

- Có hai cách nạp chồng: sử dụng phương thức của lớp và sử dụng hàm bên ngoài.

BAN HỌC TẬP
Sharing is learning

NẠP CHỒNG TOÁN TỬ

4.2.1 Cài đặt – sử dụng phương thức của lớp

– Cú pháp khai báo: `<KDLTrảVề> operator@(<DSThamSố>);`

– **Ví dụ:** câu lệnh `v1 + v2` sẽ tương đương với `v1.operator+(v2)`.

– Số lượng tham số đầu vào = Số toán hạng – 1.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

NẠP CHỒNG TOÁN TỬ

4.2.1 Cài đặt – sử dụng phương thức của lớp

Ví dụ:

```
class Vector2D {
    //…
    int x, y;
public:
    Vector2D operator+(const Vector2D& v2){
        Vector2D kq;
        kq.x = x + v2.x;
        kq.y = y + v2.y;
        return kq;
    }
}; // Đối số đầu tiên mặc định là đối tượng gọi toán tử
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

NẠP CHỒNG TOÁN TỬ

4.2.2 Cài đặt – sử dụng hàm bên ngoài

– Ví dụ: câu lệnh `v1 + v2` sẽ ứng với `operator+(v1,v2)`

– Số lượng tham đối số = số toán hạng

– Hàm bên ngoài phải là **hàm bạn** để có thể truy cập vào các thuộc tính riêng tư của lớp.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

NẠP CHỒNG TOÁN TỬ

4.2.2 Cài đặt – sử dụng hàm bên ngoài

Ví dụ:

```
class Vector2D {
//…
   friend Vector2D operator+(const Vector2D &v1, const Vector2D &v2);
};
// Đầy đủ 2 toán hạng
Vector2D operator+(const Vector2D &v1, const Vector2D &v2){
    Vector2D kq;
    kq.x = v1.x + v2.x;
    kq.y = v1.y + v2.y;
    return kq;
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

NẠP CHỒNG TOÁN TỬ

4.3.1 Chuyển kiểu

Ví dụ:

```
class PhanSo {
private:
    int tu, mau;
public:
    //Chuyển kiểu bằng constructor
    PhanSo(int a) : tu(a), mau(1){}
    PhanSo operator+(const PhanSo& ps2);
};
int main() {
    PhanSo a(2), b;
    b = a + 3; //3 sẽ được chuyển qua phân số nhờ constructor
    return 0;
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

NẠP CHỒNG TOÁN TỬ

4.3.1 Chuyển kiểu

- Lưu ý: nếu nạp chồng bằng phương thức của lớp thì toán hạng đầu tiên phải là đối tượng của lớp đó.

- Ví dụ:

```
PhanSo a(3,4);
3 + a; // Sai
```

- Câu lệnh trên sai vì không thể gọi phương thức `operator+` trên số nguyên 3.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

NẠP CHỒNG TOÁN TỬ

4.3.2 Chuyển kiểu bằng toán tử chuyển kiểu

Ví dụ:

```
class PhanSo {
private:
    int tu, mau;
public:
    //….
    operator float(){
        return (float)tu/mau;
    }
};
int main(){
    PhanSo a(3,4);
    cout<<(float)a; // 0.75 do a được chuyển kiểu sang float
    return 0 ;
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

NẠP CHỒNG TOÁN TỬ

4.3.3 Sự nhập nhằng

Sự nhập nhằng xảy ra khi lớp của bạn định nghĩa bằng constructor lẫn chuyển kiểu bằng toán tử chuyển kiểu. Cách xử lý duy nhất là chọn 1 trong 2 cách chuyển kiểu.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

NẠP CHỒNG TOÁN TỬ

4.3.3 Sự nhập nhằng

Ví dụ:

```
class PhanSo {
//...
public:
    PhanSo(int a);
    operator float();
};
int main(){
    PhanSo a(3,4);
    a + 3; // 3 thành Phân số hay a thành float?
    // -> Lỗi do sự nhập nhằng
    return 0 ;
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

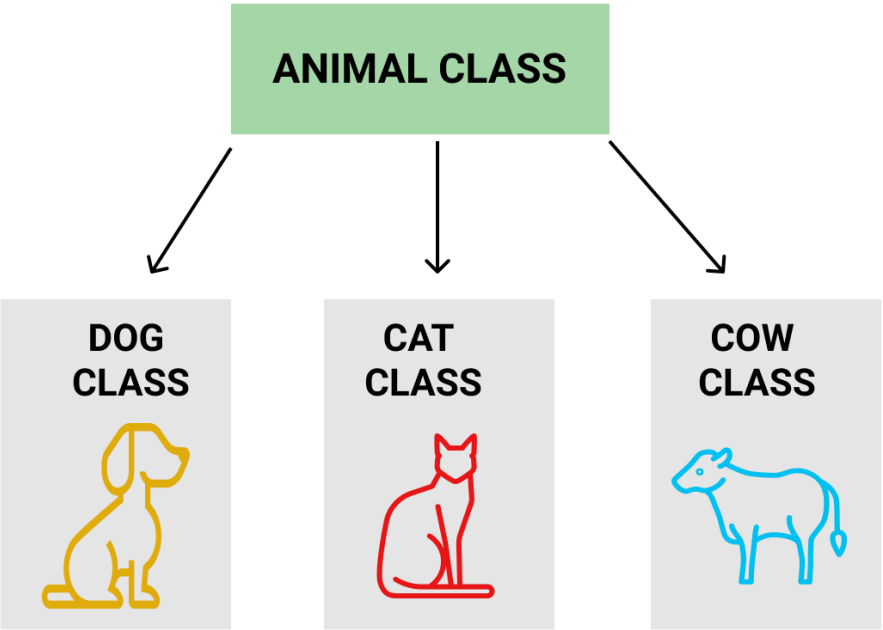
Chương 5

TÍNH KẾ THỪA

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5. KẾ THỪA

Kế thừa là một đặc điểm của ngôn ngữ lập trình dùng để biểu diễn mối quan hệ **đặc biệt hóa – tổng quát hóa** giữa các lớp (lớp này là trường hợp đặc biệt của lớp kia).



ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5. KẾ THỪA

Lợi ích của kế thừa:

+) Kế thừa cho phép **xây dựng lớp mới** từ lớp đã có.

+) Kế thừa cho phép tái sử dụng **đoạn mã,** chương trình chung, nhờ vậy có thể dễ dàng sửa chữa, nâng cấp hệ thống.

+) Định nghĩa sự tương thích giữa các lớp, nhờ đó ta có thể chuyển kiểu tự động.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5. KẾ THỪA

– Cú pháp khai báo:

```
class B : <từ khóa dẫn xuất> A {
        ...
};
```

– Lớp B kế thừa từ lớp A. B là lớp con (lớp dẫn xuất), A là lớp cha (lớp cơ sở).

– Lớp con sở hữu các **thuộc tính, phương thức** của lớp cha mà không cần phải khai báo.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5. KẾ THỪA

```
class Customer
{
    protected:
        string name;
        int numbers;
        double dongia;
    public:
        void input();
        void output();
};
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5. KẾ THỪA

```
class CustomerA : public Customer
{
    protected:
        double saleOff;
    public:
        void input();
        double total();
};
```

```
class CustomerB : public Customer
{
    protected:
        int year;
    public:
        double total();
};
```

```
class CustomerC : public Customer
{
    protected:
        double sale;
        int CID;
    public:
        double total();
};
```

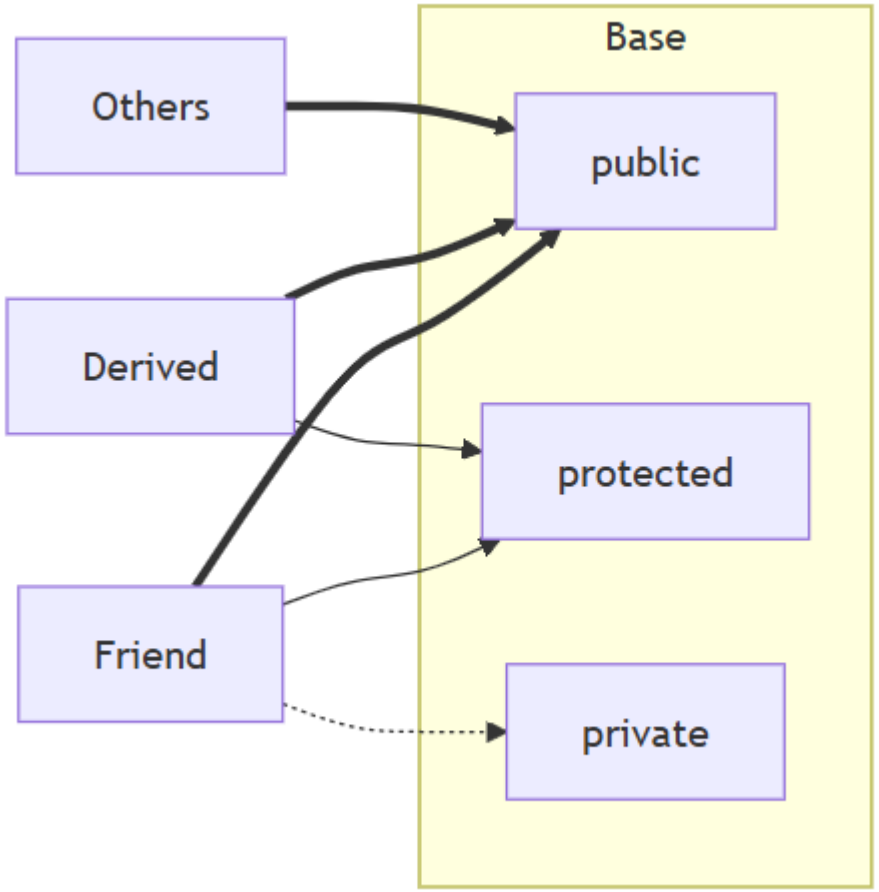
ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.1. PHẠM VI TRUY CẬP PROTECTED

- Sự kết hợp giữa `public` và `private`:

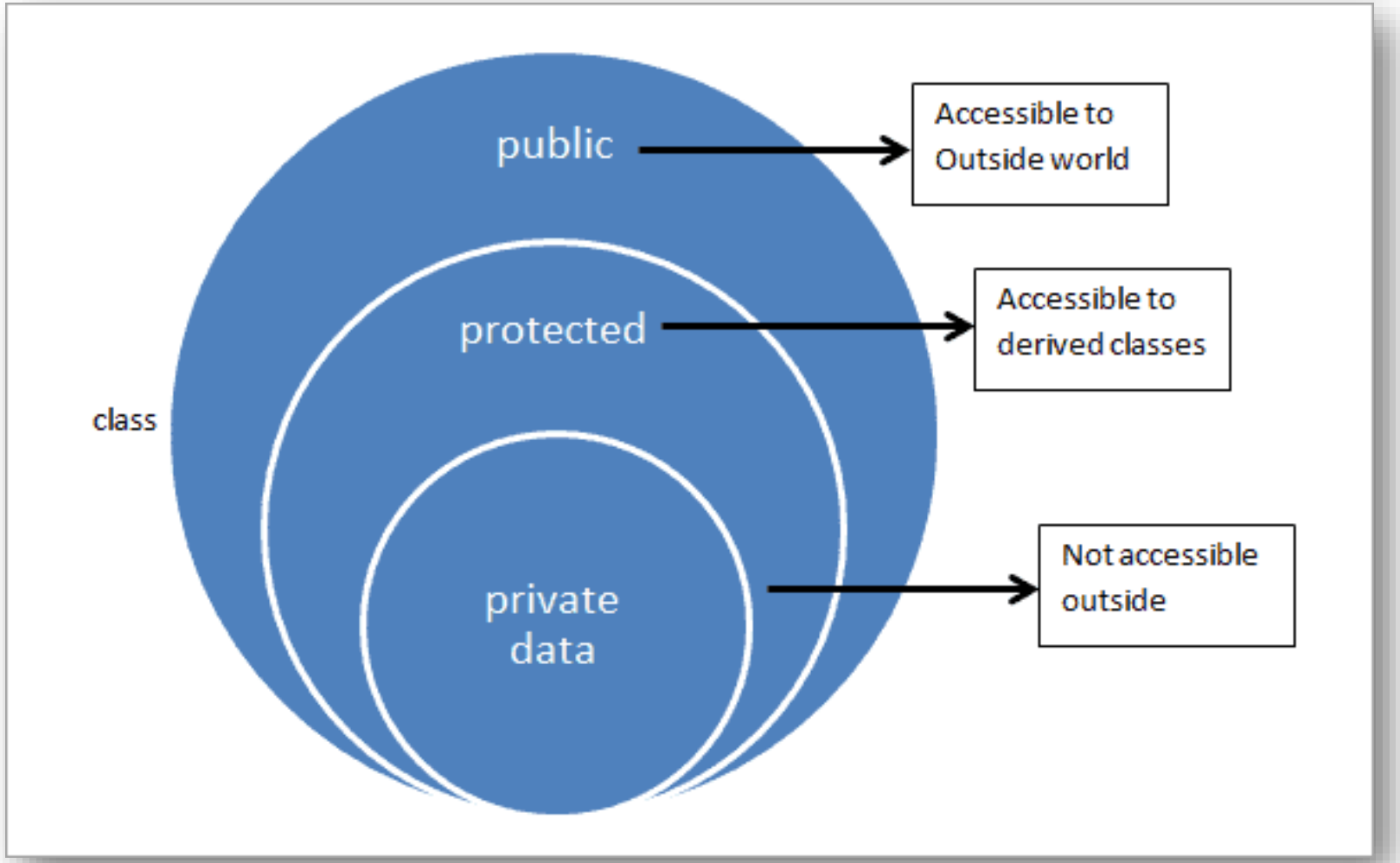
+) Giống với `private`, các thành viên `protected` không thể được truy cập từ bên ngoài lớp.

+) Giống với public, các thành viên protected của lớp cơ sở có thể được truy cập bởi bạn và các lớp dẫn xuất của nó.



Sharing is learning

5.1. PHẠM VI TRUY CẬP



BOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.2. CÁC KIỂU KẾ THỪA

Từ khóa dẫn xuất dùng để **thay đổi mức độ truy cập** của các thành viên ở lớp cha khi chúng được kế thừa xuống lớp con:

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.2. CÁC KIỂU KẾ THỪA

Phạm vi truy xuất được chỉ định ở lớp cơ sở:	Từ khóa dẫn xuất khi khai báo lớp con:	Phạm vi truy xuất ở lớp con được chuyển thành:
Public	public	public
Protected		protected
Private		**Không truy cập được**
Public	protected	protected
Protected		protected
Private		**Không truy cập được**
Public	private	private
Protected		private
Private		**Không truy cập được**

Sharing is learning

5.2. CÁC KIỂU KẾ THỪA

Phạm vi truy xuất được chỉ định ở lớp cơ sở:	Từ khóa dẫn xuất khi khai báo lớp con:	Phạm vi truy xuất ở lớp con được chuyển thành:
Public	public	public
Protected		protected
Private		**Không truy cập được**
Public	protected	protected
Protected		protected
Private		**Không truy cập được**
Public	private	private
Protected		private
Private		**Không truy cập được**

Sharing is learning

5.2. CÁC KIỂU KẾ THỪA

- Từ khóa dẫn xuất dùng để **thay đổi mức độ truy cập** của các thành viên ở lớp cha khi chúng được kế thừa xuống lớp con:

- Lớp con kế thừa kiểu `public` từ lớp cha thì các thành phần `protected` của lớp cha trở thành `protected` của lớp con, các thành phần `public` của lớp cha trở thành `public` của lớp con.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.2. CÁC KIỂU KẾ THỪA

```
class Machine
{
    public:
        int public_property = 1;
    private:
        int private_property = 2;
    protected:
        int protected_property = 3;
};
class Car : public Machine {};
//Trong class con
class Testing : public Car
{
    public:
        void test()
        {
            cout << public_property << '\n';
            cout << private_property << '\n';
            cout << protected_property << '\n';
        }
};
//Ngoài class
int main()
{
    Testing t;
    cout << t.public_property << '\n';
    cout << t.protected_property << '\n';
}
```

Sharing is learning

5.2. CÁC KIỂU KẾ THỪA

Phạm vi truy xuất được chỉ định ở lớp cơ sở:	Từ khóa dẫn xuất khi khai báo lớp con:	Phạm vi truy xuất ở lớp con được chuyển thành:
Public	public	public
Protected		protected
Private		**Không truy cập được**
Public	protected	protected
Protected		protected
Private		**Không truy cập được**
Public	private	private
Protected		private
Private		**Không truy cập được**

Sharing is learning

5.2. CÁC KIỂU KẾ THỪA

- Từ khóa dẫn xuất dùng để **thay đổi mức độ truy cập** của các thành viên ở lớp cha khi chúng được kế thừa xuống lớp con:

- Lớp con kế thừa kiểu `protected` từ lớp cha thì các thành phần `protected` và `public` của lớp cha trở thành `protected` của lớp con.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.2. CÁC KIỂU KẾ THỪA

```
3   class Machine
4   {
5       public:
6           int public_property = 1;
7       private:
8           int private_property = 2;
9       protected:
10          int protected_property = 3;
11  };
12  class Car : protected Machine {};
13  //Trong class con
14  class Testing : public Car
15  {
16      public:
17          void test()
18          {
19              cout << public_property << '\n';
20              cout << private_property << '\n';
21              cout << protected_property << '\n';
22          }
23  };
24  //Ngoài class
25  int main()
26  {
27      Testing t;
28      cout << t.public_property << '\n';
29      cout << t.private_property << '\n';
30      cout << t.protected_property << '\n';
31  }
```

5.2. CÁC KIỂU KẾ THỪA

Phạm vi truy xuất được chỉ định ở lớp cơ sở:	Từ khóa dẫn xuất khi khai báo lớp con:	Phạm vi truy xuất ở lớp con được chuyển thành:
Public	public	public
Protected		protected
Private		**Không truy cập được**
Public	protected	protected
Protected		protected
Private		**Không truy cập được**
Public	private	private
Protected		private
Private		**Không truy cập được**

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.2. CÁC KIỂU KẾ THỪA

- Từ khóa dẫn xuất dùng để **thay đổi mức độ truy cập** của các thành viên ở lớp cha khi chúng được kế thừa xuống lớp con:

- Lớp con kế thừa kiểu `private` từ lớp cha thì các thành phần `protected` và `public` của lớp cha trở thành `private` của lớp con.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.2. CÁC KIỂU KẾ THỪA

```
3   class Machine
4   {
5       public:
6           int public_property = 1;
7       private:
8           int private_property = 2;
9       protected:
10          int protected_property = 3;
11  };
12  class Car : private Machine {};
13  //Trong class con
14  class Testing : public Car
15  {
16      public:
17          void test()
18          {
19              cout << public_property << '\n';
20              cout << private_property << '\n';
21              cout << protected_property << '\n';
22          }
23  };
24  //Ngoài class
25  int main()
26  {
27      Testing t;
28      cout << t.public_property << '\n';
29      cout << t.private_property << '\n';
30      cout << t.protected_property << '\n';
31  }
```

5.2. CÁC KIỂU KẾ THỪA

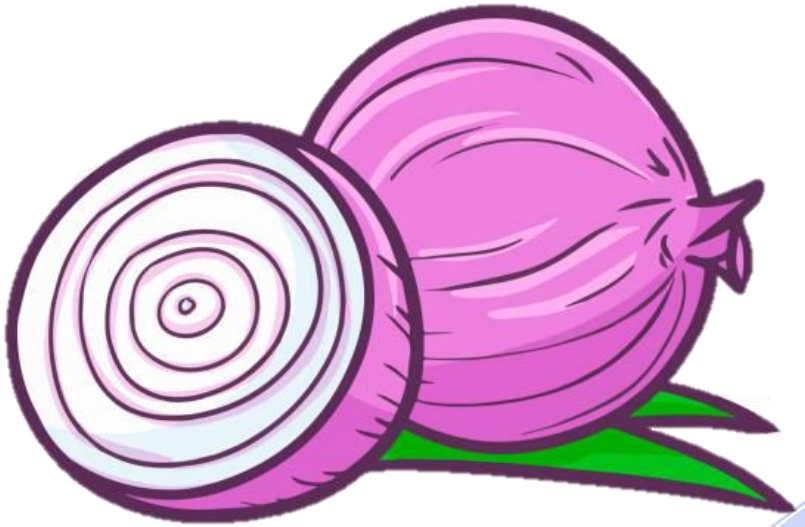
Phạm vi truy xuất được chỉ định ở lớp cơ sở:	Từ khóa dẫn xuất khi khai báo lớp con:	Phạm vi truy xuất ở lớp con được chuyển thành:
Public	public	public
Protected		protected
Private		**Không truy cập được**
Public	protected	protected
Protected		protected
Private		**Không truy cập được**
Public	private	private
Protected		private
Private		**Không truy cập được**

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.3. CONSTRUCTOR - DESTRUCTOR

5.3.1. Thứ tự gọi constructor

Khi một đối tượng lớp dẫn xuất (derived class) được tạo, **constructor của lớp cơ sở (base class)** được gọi **trước** constructor của lớp dẫn xuất.



ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.3. CONSTRUCTOR - DESTRUCTOR

5.3.1. Thứ tự gọi constructor

```
Parent constructor is calling
Child constructor is calling
Grandchild constructor is calling
```

```
class Parent
{
    public:
        Parent() {
            cout << "Parent constructor is calling" << endl;
        }
};
class Child : public Parent
{
    public:
        Child() {
            cout << "Child constructor is calling" << endl;
        }
};
class Grandchildren : public Child
{
    public:
        Grandchildren() {
            cout << "Grandchild constructor is calling" << endl;
        }
};
int main()
{
    Grandchildren c;
}
```

5.3. CONSTRUCTOR - DESTRUCTOR

5.3.1. Thứ tự gọi constructor

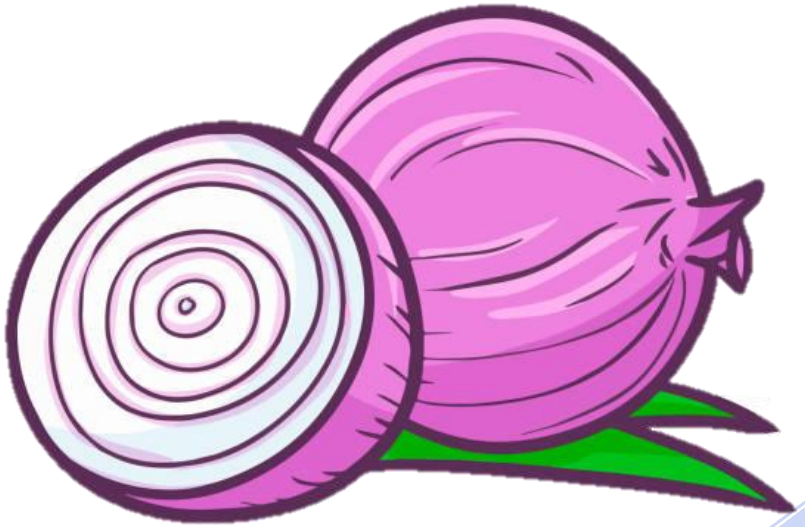
```
Parent constructor is calling
Child constructor is calling
Grandchild constructor is calling
```

```
class Parent
{
    public:
        Parent() {
            cout << "Parent constructor is calling" << endl;
        }
};
class Child
{
    public:
        Child() {
            cout << "Child constructor is calling" << endl;
        }
};
class Grandchildren : public Parent, public Child
{
    public:
        Grandchildren() {
            cout << "Grandchild constructor is calling" << endl;
        }
};
int main()
{
    Grandchildren c;
}
```

5.3. CONSTRUCTOR - DESTRUCTOR

5.3.2. Thứ tự gọi destructor

Khi một đối tượng lớp dẫn xuất **(derived class)** bị chết đi, destructor **constructor của lớp dẫn xuất (derived class)** được gọi **trước, destructor** của lớp cơ sở được gọi sau.



ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.3. CONSTRUCTOR - DESTRUCTOR

5.3.2. Thứ tự gọi dst

```
Grandchild destructor is calling
Child destructor is calling
Parent destructor is calling
```

```
class Parent
{
    public:
        ~Parent() {
            cout << "Parent destructor is calling" << endl;
        }
};
class Child : public Parent
{
    public:
        ~Child() {
            cout << "Child destructor is calling" << endl;
        }
};
class Grandchildren : public Child
{
    public:
        ~Grandchildren() {
            cout << "Grandchild destructor is calling" << endl;
        }
};
int main()
{
    Grandchildren c;
}
```

Sharing is learning

5.3. CONSTRUCTOR - DESTRUCTOR

5.3.3. Lưu ý

1. Constructor không phải là một thuộc tính hay phương thức đối tượng mà nó thuộc về chính Class của đối tượng

2. Error: lớp con cố gọi constructor của lớp cha nhưng không gọi được

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.3. CONSTRUCTOR - DESTRUCTOR

5.3.3. Lưu ý

```
class Parent
{
    public:
        Parent(int x) {
            cout << "Parent constructor is calling" << endl;
        }
};
class Child : public Parent
{
    public:
        Child() {
            cout << "Child constructor is calling" << endl;
        }
};
```

Cố gọi constructor mặc định của lớp cha nhưng không tìm thấy

BOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

5.3. CONSTRUCTOR - DESTRUCTOR

5.3.3. Lưu ý

Gọi tường minh constructor của lớp cha.

```
class Parent
{
    public:
        Parent(int x) {
            cout << "Parent constructor is calling" << endl;
        }
};
class Child : public Parent
{
    public:
        Child(int x) : Parent(x) {
            cout << "Child constructor is calling" << endl;
        }
};
int main()
{
    Child c(10);
}
```

5.4. PHÉP GÁN VỚI CON TRỎ TRONG KẾ THỪA

- Một biến con trỏ kiểu lớp cha có thể giữ địa chỉ của của một đối tượng thuộc lớp con.
- **Cú pháp:** Base* p = new Derived;
- Tuy nhiên ngược lại thì sai.
- Ví dụ:

```
Cha cha;
Con con;
Cha* chaPtr = &con; // Đúng
Con* conPtr = &cha; // Sai
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Chương 6

TÍNH ĐA HÌNH

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

6. ĐA HÌNH

Khi gọi thực hiện lời gọi một phương thức của lớp, trình biên dịch sẽ quan tâm đến **kiểu của con trỏ** chứ không phải đối tượng mà con trỏ đang trỏ tới

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

6. ĐA HÌNH

Chương trình sẽ gọi đến phương thức show() của lớp Computer, mà không gọi tới phương thức show() của lớp DELL.

Để giải quyết vấn đề này, chúng ta cần sử dụng đến tính đa hình.

```
#include <bits/stdc++.h>
using namespace std;
class Computer
{
    public:
    void show(){
        cout << "May vi tinh" << endl;
    }
};
class DELL: public Computer
{
    public:
    void show(){
        cout << "may DELL" << endl;
    }
};
int main()
{
    Computer *a = new Computer;
    a->show();
    Computer *b = new DELL;
    b->show();
}
```

```
May vi tinh
May vi tinh
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

6. ĐA HÌNH

– Ngoài việc được thừa hưởng các thuộc tính và phương thức từ lớp cha, lớp con còn có thể **định nghĩa thêm** các thuộc tính, phương thức mới và **định nghĩa lại** một vài phương thức ở lớp cha.

– **Phương thức ảo** là những phương thức lớp cha muốn lớp con định nghĩa một **phiên bản riêng** của nó.

Lưu ý: Để dùng tính đa hình thì ta phải sử dụng con trỏ (tham chiếu). Nếu chỉ sử dụng đối tượng con thì nó không thể hiện được

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
Sharing is learning

6. ĐA HÌNH

```cpp
#include <bits/stdc++.h>
using namespace std;
class Machine
{
    public:
        void show() {
            cout << "Coumputer!!!!!!!!!!";
        }
};
class Asus : public Machine
{
    public:
        void show() {
            cout << "Asussssssssss";
        }
};
int main()
{
    Machine c;
    c.show();
}
```

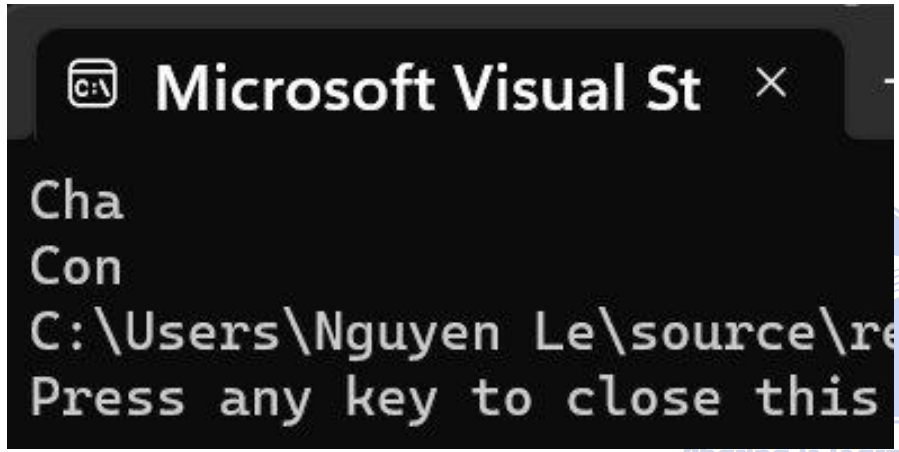
```cpp
class Machine
{
    public:
        virtual void show()
        {
            cout << "Computer!!!!!!!";
        }
};
class Asus : public Machine
{
    public:
        void show() override
        {
            cout << "Asussssssssssss";
        }
};
int main()
{
    Machine *c = new Asus;
    c->show();
}
```

Sharing is learning

6. ĐA HÌNH

Khi gọi một **phương thức ảo** thông qua một **con trỏ** kiểu Base, tùy thuộc vào **kiểu dữ liệu** mà biến con trỏ đang giữ địa chỉ, phiên bản của phương thức ảo trong **lớp tương ứng** sẽ được thực hiện

```
Cha base;
Con derived;
Cha* basePtr;
basePtr = &base;
basePtr->Xuat();
basePtr = &derived;
basePtr->Xuat();
```



6. ĐA HÌNH

Khi khai báo hàm ảo với từ khóa **virtual** nghĩa là hàm này sẽ được gọi theo loại đối tượng được lưu địa chỉ

+) Nếu không dùng từ khóa **virtual** trình biên dịch sẽ gọi hàm tại lớp cơ sở

+) Nếu dùng từ khóa **virtual** trình biên dịch sẽ gọi hàm tại lớp dẫn xuất

```
class Machine
{
    public:
        void show() {
            cout << "Coumputer!!!!!!!!!!";
        }
};
class Asus : public Machine
{
    public:
        void show() {
            cout << "Asussssssssssss";
        }
};
int main()
{
    Machine *c = new Asus;
    //gọi tới hàm show() của lớp Asus
    c->show();
}
```

Sharing is learning

6. ĐA HÌNH

Lưu ý 1

Khi gọi phương thức thông qua con trỏ lớp cha, ta chỉ có thể **gọi các phương thức mà lớp cha sỡ hữu**, cho dù con trỏ đang giữ địa chỉ của đối tượng thuộc lớp cha hay con.

– Ví dụ:

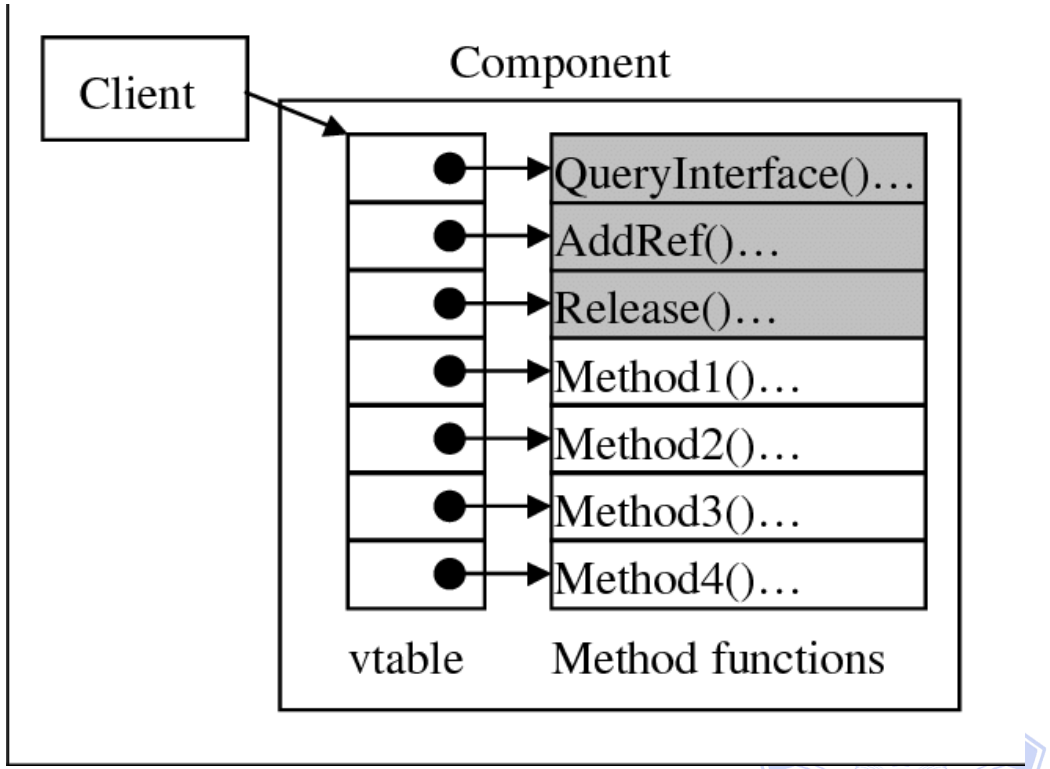
```
Con derived;
// PhuongThucCon do lớp Con định nghĩa thêm
derived.PhuongThucCon();  // Đúng
Cha* basePtr = &derived;
basePtr->PhuongThucCon(); // Sai!
```

BOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

6. ĐA HÌNH

Lưu ý 2

- Khi nhận thấy có khai báo **virtual** trong Base class, trình biên dịch sẽ tạo ra 1 bảng gọi là vtable (virtual table). vtable là nơi lưu các con trỏ, cho biết đến hàm ảo thực tế là gì?
- Mỗi đối tượng kế thừa từ **Base** sẽ có một con trỏ đặc biệt, gọi là *vpointer*, lưu giữ địa chỉ ô nhớ của hàm override tương ứng trong vtable.



BAN HỌC TẬP
Sharing is learning

6. ĐA HÌNH

Lưu ý 2

- Lỗi "undefined reference to vtable for ClassName" thường xảy ra khi: Bạn **khai báo một hàm ảo** trong **Base class**, nhưng **không cung cấp định nghĩa (implementation)** cho nó
- Để sửa lỗi này, ta thêm { } hoặc = 0 ngay sau hàm **virtual** ở Base class hoặc

```
✗ cd "/home/cloud/workspace/MyProject/LamTaiLieu/GiaiDeOOP7" && g++ tamli.cpp -o tamli && "/home/cloud/work
/usr/bin/ld: /tmp/ccTZCW3F.o: warning: relocation against `_ZTV5TamLi' in read-only section `.text._ZN5TamLiC2
/usr/bin/ld: /tmp/ccTZCW3F.o: in function `TamLi::TamLi()':
tamli.cpp:(.text._ZN5TamLiC2Ev[_ZN5TamLiC5Ev]+0xb): undefined reference to `vtable for TamLi'
/usr/bin/ld: /tmp/ccTZCW3F.o:(.data.rel.ro._ZTI1N[_ZTI1N]+0x10): undefined reference to `typeinfo for TamLi'
/usr/bin/ld: /tmp/ccTZCW3F.o:(.data.rel.ro._ZTI1A[_ZTI1A]+0x10): undefined reference to `typeinfo for TamLi'
/usr/bin/ld: /tmp/ccTZCW3F.o:(.data.rel.ro._ZTI1E[_ZTI1E]+0x10): undefined reference to `typeinfo for TamLi'
/usr/bin/ld: /tmp/ccTZCW3F.o:(.data.rel.ro._ZTI1C[_ZTI1C]+0x10): undefined reference to `typeinfo for TamLi'
/usr/bin/ld: /tmp/ccTZCW3F.o:(.data.rel.ro._ZTI1O[_ZTI1O]+0x10): undefined reference to `typeinfo for TamLi'
/usr/bin/ld: warning: creating DT_TEXTREL in a PIE
collect2: error: ld returned 1 exit status
```

```cpp
class TamLi
{
    protected:
        int c;
    public:
        void setChiSo(int c) {
            this->c = c;
        }
        int getChiSo() {
            return this->c;
        }
        virtual string outputRes();
};
```

BAN HỌC TẬP
Sharing is learning

6. ĐA HÌNH

Lưu ý 3

- Phân biệt giữa nạp chồng (overload) và ghi đè (override):

- **Nạp chồng:** Các phương thức có **cùng tên** nhưng danh sách **tham số khác nhau**, kiểu dữ liệu trả về có thể giống hoặc khác nhau.

- **Ghi đè:** Các phương thức **giống nhau hoàn toàn** về tên, danh sách tham số, kiểu dữ liệu trả về.

BAN HỌC TẬP
Sharing is learning

6. ĐA HÌNH

Lưu ý 3

```cpp
class MyClass {
public:
    void display() {
        cout << "display() không có tham số\n";
    }
    void display(int x) {
        cout << "display(int): " << x << '\n';
    }
    void display(double y) {
        cout << "display(double): " << y << '\n';
    }
};

int main() {
    MyClass obj;
    cout << "--- Nạp chồng display() ---\n";
    obj.display();          // display() không có tham số
    obj.display(10);        // display(int)
    obj.display(3.14);      // display(double)

    return 0;
}
```

```cpp
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() {
        cout << "Base::show()\n";
    }
};

class Derived : public Base {
public:
    void show() override { // Ghi đè hàm show() của Base
        cout << "Derived::show()\n";
    }
};

int main()
{
    Base b;
    Derived d;
    Base* ptr;
    cout << "--- Gọi show() từ con trỏ Base ---\n";
    ptr = &b;
    ptr->show(); // gọi Base::show()
    ptr = &d;
    ptr->show(); // gọi Derived::show() vì có override
    return 0;
}
```

6. ĐA HÌNH

Lưu ý 4

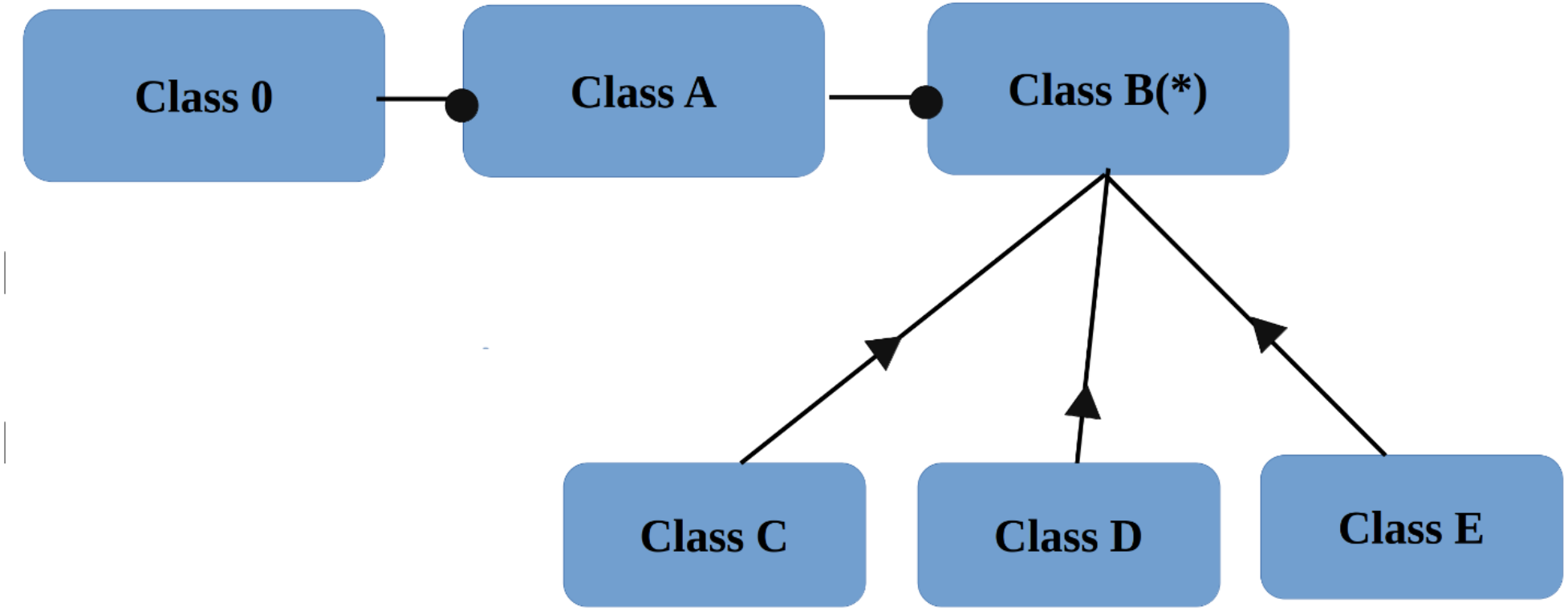
- Ta có thể sử dụng hàm **dynamic_cast** để kiểm tra con trỏ đó đang giữ địa chỉ của đối tượng con nào, cũng như ép nó về con trỏ thuộc kiểu đối tượng con đó

- **Cú pháp:** dynamic_cast<DerivedClass*>(pointer)

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

6. ĐA HÌNH

Lưu ý 5: Mô hình thiết kế



Sharing is learning

6. ĐA HÌNH

```
class Community
{
    protected:
        vector<Person*> a;
        int n;
    public:
        void input();
        void output(int);
        void dangerPeron();
};
```

```
class Person
{
    protected:
        vector<TamLi*> t;
    public:
        void input();
        void output();
        void saveData(string);
        bool isDanger();
};
```

BAN HỌC TẬP
Sharing is learning

6. ĐA HÌNH

```cpp
class TamLi
{
    protected:
        int c;
    public:
        void setChiSo(int c) {
            this->c = c;
        }
        int getChiSo() {
            return this->c;
        }
        virtual string outputRes() {};
};
```

```cpp
class O : public TamLi
{
    public:
        string outputRes() override
        {
            if (c <= 30 && c >= 0) return "Truyen danh gia O thap";
            else if (c >= 70) return "Truyen danh gia O cao";
            return "Khong xac dinh ro";
        }
};
```

```cpp
class C : public TamLi
{
    public:
        string outputRes() override
        {
            if (c <= 30 && c >= 0) return "Truyen danh gia C thap";
            else if (c >= 70) return "Truyen danh gia C cao";
            return "Khong xac dinh ro";
        }
};
```

Sharing is learning

BAN HỌC TẬP CÔNG NGHỆ PHẦN MỀM

TRAINING CUỐI KỲ HỌC KỲ II NĂM HỌC 2024 – 2025

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

Sharing is learning

HẾT

CẢM ƠN CÁC BẠN ĐÃ THEO DÕI
CHÚC CÁC BẠN CÓ KẾT QUẢ THI THẬT TỐT!

BAN HỌC TẬP

Khoa Công nghệ Phần mềm

Trường Đại học Công nghệ Thông tin

Đại học Quốc gia thành phố Hồ Chí Minh

CONTACT

bht.cnpm.uit@gmail.com

fb.com/bhtcnpm

fb.com/groups/bht.cnpm.uit

TEAM TIẾNG ANH

english.with.bht@gmail.com

creative.owl.se

english.with.bht

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
HỌC THUẬT

- Training
- Giải đáp
- Chia sẻ

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
TRUYỀN THÔNG

- Design ấn phẩm
- Viết content
- Chụp ảnh

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
TIẾNG ANH

- Instagram
- Dịch thuật
- Thi thử